AI TECHNOLOGY STACK RECOMMENDATION

Lab 2 deliverable 6 (procedure §3.4). Version 0.1, 2026-09-02. **Export this file to PDF before submission** — §4 of the lab manual asks for a PDF.

> **Naming note.** The application was renamed **D-Fence** on 2026-09-03, after this exercise was run. The attachment filenames and the AI response above are left exactly as they were at the time — this section is a record of what happened, not a description of the current model.

---

1. What was submitted to the AI

AI tool	Claude (Opus 5), via the Claude Code interface
Date	2026-09-02
Attachment 1	`REQUIREMENTS.md` v0.3 — the system description, 349 atomised requirements
Attachment 2	`DengueGuard-EntityClasses.png` — the entity class diagram, 23 entity classes
Attachment 3	`DengueGuard-BoundaryControl.png` — boundary and control classes
Prompt (verbatim, as specified in lab §3.4.2)	"*Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team.*"

A disclosure that belongs at the top of this report. The team is not choosing a stack from nothing. `REQUIREMENTS.md` already names the Telegram Bot API for alerts and OneMap for geocoding and map tiles, because those choices were made during requirements work and constrain the requirements themselves. Supabase was chosen earlier still, in `BACKLOG.md` (F2), and is *not* named in `REQUIREMENTS.md` — an earlier version of this sentence said it was, which was wrong and is corrected here (2026-09-03). This report therefore does two things: it records the AI's recommendation as the lab asks, and it evaluates that recommendation against decisions the team had already taken. Where the AI agrees, that is corroboration; where it differs, §3 says which we are following and why.

---

2. The AI's response (verbatim)

> **Recommended stack**
>
> **Front end — React with TypeScript, built with Vite.** The screen inventory has 24 screens across three roles with substantially different views, which is more structure than a template-driven approach handles comfortably. TypeScript matters here more than usual: your entity model has 13 enumerations with closed value sets (report status, work-order status, priority tier), and a typed union catches an invalid state transition at compile time rather than in the Lab 5 demo.

> **Mapping — Leaflet with the OneMap tile layer.** Your requirements need polygon rendering, layer toggling, click-to-select and marker overlays — all core Leaflet. Do not reach for a heavier GIS library; nothing in §9 requires it.
>
> **Back end — Node.js with Express and TypeScript, or Python with FastAPI.** Either is defensible. Node lets one language cover both ends, which for a five-person team with mixed experience reduces the number of people who cannot help with a given ticket. FastAPI is better if the scoring engine grows numerically demanding, because the normalisation and weighting work is more natural in Python with NumPy.
>
> **Database — PostgreSQL with PostGIS.** This is the one place I would push back on any alternative. Requirement 3.1.8 evaluates saved locations against cluster boundaries on every ingestion cycle, 5.1.7 associates reports with containing clusters, and 1.2.5 assigns nearest stations by great-circle distance. Those are point-in-polygon and nearest-neighbour queries. Doing them in application code over a non-spatial store means re-implementing a spatial index badly. PostGIS makes each of them a single query.
>
> **Auth and storage — Supabase.** Postgres with PostGIS available, plus authentication, row-level security and file storage for report and completion photographs. Row-level security maps almost directly onto your §2.3 access-control requirements. For an entry-level team this removes about a week of undifferentiated work.
>
> **Scheduling — node-cron or APScheduler in-process for development; a hosted scheduler for deployment.** You have five recurring jobs at four different intervals (rainfall every 5 minutes, clusters hourly, forecast every 6 hours, token every 48 hours). Do not hand-roll a timing loop.
>
> **Notifications — Telegram Bot API.** Already in your requirements, and the right choice: no per-message cost, no sender verification, and a five-line integration. Email would need a deliverability story you do not have time for.
>
> **Testing — Vitest or Jest for unit tests, Playwright for one end-to-end path.** Your Lab 4 deliverables name equivalence-class, boundary-value and basis-path testing of a control class, so a unit-test framework is not optional.
>
> **What I would avoid.** Do not add a state-management library until you feel the pain — React's own state plus a query cache covers this application. Do not use an ORM's automatic migrations as your schema source of truth; write the migrations by hand so the entity diagram and the database stay in step. Do not build a mobile app: the crew screens are three views and a camera upload, which a responsive web page handles.

---

3. Critical evaluation against project scope

Lab §3.4.3. Each recommendation was checked against the requirements and the module's constraints rather than accepted on the AI's authority.

Recommendation	Assessment
React + TypeScript	**Accept.** The reasoning about typed enumerations is sound and matches the 13 closed value sets in the data dictionary.
Leaflet + OneMap tiles	**Accept.** Matches the OneMap decision already recorded, and §9 needs nothing heavier.
Node/Express **or** FastAPI	**Accept, choosing Node.** The AI declined to choose; the team must. One language across both ends is worth more to a five-person team than NumPy convenience, and the scoring engine is a weighted sum over seven drivers, not heavy numerics.
PostgreSQL + PostGIS	**Accept, and the strongest point in the response.** It correctly identified that three separate requirements are spatial queries. This was not stated as a requirement anywhere; the AI inferred it from 1.2.5, 3.1.8 and 5.1.7.
Supabase	**Accept — corroboration, not new information.** Already chosen. The row-level-security observation adds a reason we had not written down.
Scheduler library	**Accept.** Five jobs at four intervals is the case for it.
Telegram Bot API	**Accept.** Already chosen.
Vitest/Jest + Playwright	**Accept.** Lab 4 requires unit-testable control classes.
"Avoid a state-management library"	**Accept with a caveat.** Reasonable, but the operations dashboard holds live-updating shared state across panels; revisit if prop-passing becomes unmanageable.
"Avoid ORM auto-migrations"	**Accept.** Hand-written migrations keep the entity diagram authoritative, which the module grades.
"Do not build a mobile app"	**Accept.** Also consistent with the standing constraint that substantial hand-coded implementation is required — a low-code mobile builder would weaken the submission.

One thing the AI did not raise, and should have. Nothing in its answer addressed the module's constraint that low-code tooling is acceptable only for peripheral modules. A recommendation that had leaned on a low-code back end would have been actively harmful to the grade, and the AI had no way to know that because it is a course rule, not a technical one. The lesson is the same one the Lab 1 critique produced: an AI answers the question asked, against the constraints it can see.

---

4. Evaluation of one non-language technology (lab §3.4.4)

Technology selected: PostGIS.

PostGIS is a spatial extension to PostgreSQL that adds geometry types and spatial indexing. It suits this project because three of our requirements — nearest rainfall stations (1.2.5), saved locations against cluster boundaries (3.1.8) and reports bound to a containing cluster (5.1.7) — are point-in-polygon and nearest-neighbour problems it answers in one indexed query, where the alternative is re-implementing a spatial index in application code. Its cost to an entry-level team is small: three functions (`ST_Contains`, `ST_Distance`, `ST_DWithin`) and an extension Supabase can enable without extra infrastructure.

---

5. Selected stack

Layer	Choice
Front end	React + TypeScript, Vite
Mapping	Leaflet with OneMap tiles
Back end	Node.js + Express + TypeScript
Database	PostgreSQL + PostGIS, via Supabase
Auth, storage	Supabase (row-level security, photo storage)
Scheduling	node-cron in development
Notifications	Telegram Bot API
Testing	Vitest, Playwright for one end-to-end path

---

6. Declaration of AI use

Claude (Opus 5) was used to produce the recommendation in §2, as directed by lab procedure §3.4.2. Every recommendation was evaluated against the requirements before acceptance (§3), one point was identified as a gap in the AI's answer rather than in ours, and the choice left open by the AI — Node versus FastAPI — was made by the team. The stack in §5 is the team's decision and every item in it is understood by the members who selected it.